

ASSIGNMENT

Mock Storefront — Product Search & Scheduled Price Tracker

Deadline: September 27, 2026 (Sunday) — 1:00 PM IST

Target storefront: <https://demo.inelabteamdev.com/>

1. Objective

A hosted mock storefront is provided for this assignment. Your task is to build a small web application that lets a user search for and pick a product from the hosted mock store, by partial or full product name, and then track its price over time by scraping the store on a schedule. All products come in several options (for example storage size, kit, or pack size), each with its own price. The user picks both the product and the option to track. Any other information about the products may also be provided on your dashboard.

The store is deliberately awkward to scrape, and getting the scraping to work reliably is the heart of this assignment, not the interface around it. What matters is that the scraper keeps working correctly over many unattended runs.

2. What Users Can Do

Build a small full-stack web application where users can:

- Search for and pick a product from the hosted mock store, by partial or full product name.
- Track that product, so the app scrapes its current price and stock on a fixed schedule.
- View the product's price and stock history over time as a chart or table.
- See a per-product scrape log showing each scrape attempt and its outcome.

3. Tech Stack

- **Frontend:** React.js or Vue.js, deployed on Vercel
- **Backend:** Node.js (Express) or Django, deployed on Render
- **Database:** Supabase (PostgreSQL) for tracked products and price history
- **Scraping:** lightweight HTTP fetching with HTML parsing, or a headless browser (Playwright or Puppeteer) where the page genuinely requires JavaScript rendering
- **Scheduling:** an external cron service (for example cron-job.org) or a scheduled function, since free-tier backends sleep

4. Core Features

4.1 Product Selection

- Users search the hosted mock store by partial or full product name and pick a product to track. Persist tracked products in Supabase.
- Your live dashboard must have at least 2–3 products tracked at the time of submission. The price history and scrape log should reflect real unattended runs.

4.2 Export

Provide an **Export** button on your dashboard that downloads the full scrape history as a CSV file, with one row per scrape attempt containing: the store's product ID (as shown in the product page URL), product name, selected option, timestamp (ISO 8601, UTC), price, stock, and outcome (success,

retried, or failed). Failed attempts must be included, with price and stock left empty.

4.3 Scheduled Scraping (the core challenge)

The app scrapes each tracked product's current price and stock from the mock store on a fixed schedule of **once every 2 hours**. The mock store is intentionally difficult: prices change frequently, some content loads asynchronously after a short delay, and responses are occasionally slow or return errors. Your scraper must be reliable across many unattended runs. Handle slow and failed loads with retries, recover when a request errors or a page shifts, and never silently stop or store incorrect data.

Because free-tier backends sleep, trigger scheduled scrapes via an external cron service or a scheduled function rather than an always-on loop. Prefer lightweight HTTP fetching and HTML parsing where possible. Reach for a headless browser only where the page genuinely requires it.

4.4 Price History and Scrape Log

Show each tracked product's price and stock over time as a chart or table. Show a per-product scrape log listing every scrape attempt with a timestamp and its outcome (success, retried, or failed). Failures must be recorded honestly, not hidden. You may also display extra information about the product on your dashboard according to your judgement.

4.5 Observable (Headed) Run

Provide a way to run the scraper in headed mode so its behavior can be watched. Submit a short screen recording of a headed run against the mock store, including how the scraper handles a slow or failing response.

5. Deliverables

- A link to the hosted, live site.
- Public GitHub repository URL containing all the source files.
- A short screen recording (2 to 4 minutes) of the scraper running in headed mode against the mock store, showing how it handles slow or failing responses.
- A README with setup instructions, the scraping schedule, and the environment variables required.
- A short design note explaining how you made the scraping reliable, what trade-offs you made, and what your AI tools got wrong on the first attempt and how you corrected it.
- PDF resume.

6. Evaluation Criteria

- **Scraping Reliability:** the scraper keeps working across many unattended runs, with retries and graceful failure handling. This is the core of the assessment.
- **Correctness under Difficulty:** it extracts the right price and stock even when content loads late or a response is slow, and it never stores wrong or empty data on failure.
- **Honest History and Logging:** the price history and scrape log accurately reflect what happened, including failures.
- **Judgment:** a sensible choice between lightweight fetching and a headless browser, and correct handling of the free-tier scheduling constraint.
- **Deployment:** frontend on Vercel, backend on Render, data on Supabase, reachable from the live link.

7. Bonus

- Price-drop or back-in-stock alerts, in-app or by email via SendGrid.

- A dashboard across multiple tracked products or extra information about tracked products.
- Change detection that flags when the store's page structure changes.
- Configurable scrape frequency per product.
- Option to scrape multiple product options for the same product in one scrape run.
- CI/CD with GitHub Actions.

8. Resources (Free Tiers Only)

- **Frontend hosting:** Vercel
- **Backend hosting:** Render.com
- **Database:** Supabase (PostgreSQL)
- **Scheduled scrape triggers:** cron-job.org
- **Headless browser (if needed):** Playwright or Puppeteer
- **Mock store (target site):** <https://demo.inelabteamdev.com>

9. Notes

Free-tier instances may automatically go idle or sleep. Trigger scrapes via an external cron service rather than an always-on process, and keep the instance warm if needed.

Scrape only the provided mock store. Do not scrape real retailers or third-party sites.